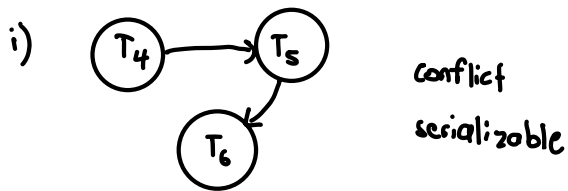


## Question 2



ii) The precedence graph is conflict serializable as it does not contain any loop

| iii) | $T_4$                                                                                                              | $T_5$                                                                                                                     | $T_6$                                                                                                               |
|------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|      | <pre>begin x = lock(P) read-item(P) s-lock(Q) read-item(Q) P = (P * S) + Q write-item(P) unlock(P) &amp; (Q)</pre> | <pre>begin s-lock(Q) read-item(Q) s-lock(R) read-item(R) WAIT x-lock(Q) Q = Q + R write-item(Q) unlock(Q) &amp; (R)</pre> | <pre>begin s-lock(R) read-item(R) WAIT WAIT WAIT WAIT WAIT WAIT x-lock(R) R = R - 100 write-item(R) unlock(R)</pre> |

### Question 3

| b) iii) | T11            | T12             | T13            | T14            |
|---------|----------------|-----------------|----------------|----------------|
|         | begin          | begin           | begin          | begin          |
|         | $\pi\_lock(A)$ | $\pi\_lock(B)$  | $\pi\_lock(A)$ | $\pi\_lock(C)$ |
|         | read(A)        | read(B)         | WAIT           | read(C)        |
|         | $A = A + 200$  | $B = B - 50$    | ⋮              | $C = C + 10$   |
|         | write(A)       | write(B)        | ⋮              | write(C)       |
|         |                | $\pi\_lock(B)$  | Read(A)        |                |
|         |                | WAIT            | $A = A - 30$   |                |
|         |                | ⋮               | write(A)       |                |
|         |                |                 | unlock(A)      |                |
|         | unlock(A)      | read(A)         |                | unlock(C)      |
|         |                | $A = A * 2$     |                |                |
|         |                | write(A)        |                |                |
|         |                | unlock(A) & (B) |                |                |

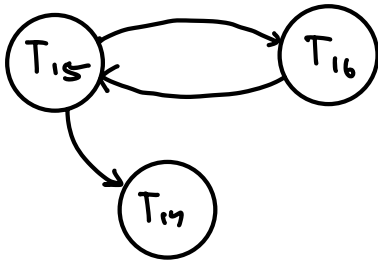
iv) Cascading rollback

#### Question 4

i) - Non-serial schedule

- Because it allows multiple transaction to be interleaved

ii)



iii) - Non conflict serializable

- Because there is a loop in the precedence graph

iv) - Last updates

- T16 writes K at  $t_5$  and T15 writes again at  $t_1$

| v) | T15             | T16       | T17        |
|----|-----------------|-----------|------------|
|    | begin           |           |            |
|    | x_lock(J)       | begin     |            |
|    | read(J)         | x_lock(K) |            |
|    | J = J + 10      | read(K)   | begin      |
|    | write(J)        | K = K - 5 | x_lock(J)  |
|    | x_lock(K)       | write(K)  | WAIT       |
|    | WAIT            | unlock(K) | :          |
|    | WAIT            |           |            |
|    | read(K)         |           | ---        |
|    | K = K + 20      |           | :          |
|    | write(K)        |           |            |
|    | unlock(J) & (K) |           | ---        |
|    |                 |           | :          |
|    |                 |           |            |
|    |                 |           | read(J)    |
|    |                 |           | J = J - 15 |
|    |                 |           | write(J)   |
|    |                 |           | unlock(J)  |